

PRACTICAL INFRASTRUCTURE AUTOMATION

SERVER TRIAGE PACK

Build a read-only PowerShell toolkit that turns scattered first-pass checks into one consistent operational report.

BUILD GUIDE

Complete project setup

USAGE GUIDE

Local, remote, and fleet runs

SAFETY FIRST

Evidence without
remediation

RHYAN JACK & GPT Sol 5.6

Version 1.0 | 22 August 2026

01

Start here

A guided build for engineers who want a repeatable first-pass server assessment.

The result

One command gathers a Windows Server snapshot and produces an HTML dashboard, CSV fleet summary, JSON evidence, run manifest, and execution log.

Best for

2nd-line and 3rd-line support, infrastructure operations, incident triage, health checks, and automation portfolios.

Time to first report

About five minutes for a local test. Remote use depends on existing WinRM and permission design.

Guide map

Pages	Section	Outcome
3-6	Design and safety	Understand the workflow, boundaries, and prerequisites
7-10	Build and configure	Validate the pack and tailor thresholds and targets
11-14	Run it	Use local, remote, CSV, credential, and HTTPS modes
15-18	Read the evidence	Interpret checks, health states, and output files
19-22	Operate it	Troubleshoot, test, harden, and extend safely

Companion source pack

This PDF is the walkthrough. The supplied ZIP contains the complete script, configuration, test file, fictional example output, README, changelog, and MIT licence.

02

What you are building

A small project with useful operational outputs, not a loose collection of console commands.

Collect once

Use the same checks for one local host, one remote server, or a CSV-defined group.

Review quickly

Surface configured warnings and critical findings without hiding the raw evidence.

Reuse the data

Feed the CSV or JSON into incident records, trend reports, dashboards, or later automation.

Stay read-only

The collector performs no restart, cleanup, installation, registry change, or remediation.

The output set

File	Audience	Purpose
Server-Triage-Report.html	Engineer	Portable dashboard with findings and detailed sections
Server-Triage-Summary.csv	Team lead / analyst	One row per target for filtering and comparison
json\.json	Automation	Structured host evidence for reuse or run comparison
run-manifest.json	Audit / support	Run ID, dates, targets, configuration, and output list
trriage-run.log	Operator	Sequence, warnings, failures, and output locations

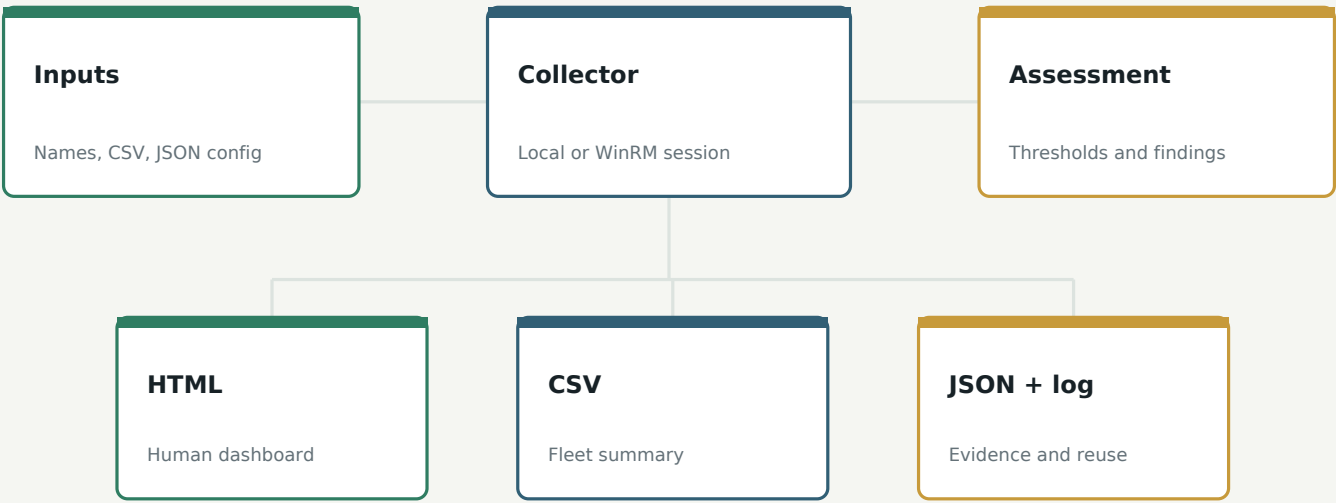
The intended use

- Run it at the start of an incident to gather a consistent evidence baseline.
- Use it for planned server reviews, support handoffs, or pre-change checks.
- Treat every threshold as a prompt for investigation, not proof of root cause.

03

Architecture and data flow

The collector, assessment logic, and presentation layer are deliberately separated.



How one run works

Stage	Action	Why it matters
1. Resolve	Read names directly or import ComputerName, Environment, and Owner from CSV.	Creates a clean, deduplicated target list.
2. Collect	Run the same script block locally or through Invoke-Command.	Keeps local and remote results consistent.
3. Assess	Compare raw values with JSON thresholds and required services.	Makes the policy visible and editable.
4. Present	Generate HTML, CSV, JSON, manifest, and log files.	Supports people, automation, and evidence retention.

Design pattern
Collection and remediation remain separate. That makes the triage pack safer to run during an incident and easier to approve for wider use.

04

Safety model

Evidence gathering should not become an unreviewed change mechanism.

The script is read-only by design

- No Restart-Service, Stop-Service, Remove-Item, registry writes, module installation, or configuration changes.
- A failed ping is informational. The script still attempts remoting because ICMP may be blocked.
- Credentials are accepted as PSCredential objects and are not written into reports or logs.
- Every run creates a new timestamped folder instead of overwriting earlier evidence.
- Section-level failures are recorded while other checks continue where possible.

Important

The output can contain hostnames, IP addresses, service-account names, listening ports, and event text. Store it as operational data and sanitise it before external sharing.

What still needs human judgement

Signals

Low disk, old patch dates, pending reboot, stopped services, long uptime, event errors, and point-in-time resource values.

Decisions

Whether the signal is expected, causal, safe to remediate, covered by change control, and owned by the right team.

Safe operating rule

Collect first. Correlate second. Change only through an approved remediation path.
This pack intentionally ends before the change step.

05

Prerequisites

No third-party modules are required, but remote collection depends on the environment.

Collection computer

- Windows 10, Windows 11, or Windows Server.
- Windows PowerShell 5.1 or PowerShell 7 on Windows.
- Write access to the chosen output folder.
- Network route to the target servers.

Remote targets

- Supported Windows Server with WMI/CIM.
- PowerShell remoting already enabled.
- WinRM allowed through the relevant firewall profile.
- An identity permitted to use the endpoint and read the requested data.

No installation step

Built-in commands only

The collector uses Windows PowerShell remoting, CIM, Get-Service, Get-WinEvent, Get-HotFix, registry reads, and built-in network cmdlets.

Permission approach

- Start in a lab using an account that can read the required data.
- For production, prefer delegated read rights or a constrained endpoint over routine use of highly privileged accounts.
- Do not embed passwords in scripts, CSV files, JSON, scheduled-task arguments, or documentation.
- Use WinRM HTTPS where the trust boundary or network design requires it.

Change boundary

Enable-PSRemoting changes the target's management configuration and firewall rules. The triage script never runs it for you.

06

Extract and validate the project

Start with the complete pack and prove that its files and configuration are intact.

PROJECT STRUCTURE

```
Server-Triage-Pack
|-- Invoke-ServerTriage.ps1
|-- Start-Local-Triage.cmd
|-- Config
|   |-- triage.config.json
|   '-- servers.example.csv
|-- Examples
|-- Tests
|   '-- Test-ServerTriagePack.ps1
|-- Output
|-- README.md
'-- LICENSE.txt
```

Step 1 - Place it in a controlled folder

POWERSHELL

```
Set-Location C:\Tools\Server-Triage-Pack
```

Step 2 - Review and unblock trusted files

AFTER REVIEWING THE SOURCE

```
Get-ChildItem -Path . -Filter *.ps1 -Recurse | Unblock-File
```

Step 3 - Run the included validation

VALIDATION

```
.\Tests\Test-ServerTriagePack.ps1
```

Expected result

The test checks required files, PowerShell parser errors, JSON threshold relationships, CSV schema, and selected mutating commands. It should finish with: All Server Triage Pack validation checks passed.

07

Configure the thresholds

The JSON file keeps operational policy outside the code.

CONFIG\TRiage.CONFIG.JSON

```
{
  "EventLookbackHours": 24,
  "MaxEventsPerLog": 20,
  "DiskWarningPercentFree": 20,
  "DiskCriticalPercentFree": 10,
  "MemoryWarningPercentFree": 15,
  "UptimeWarningDays": 45,
  "PatchWarningAgeDays": 35,
  "PatchCriticalAgeDays": 60,
  "CollectListeningPorts": true,
  "MaxListeningPorts": 40
}
```

Setting	What it controls	Review question
Disk thresholds	Free percentage on each fixed disk	Do small and large volumes need different operational policy?
Patch age	Latest dated result returned by Get-HotFix	Which platform remains the source of truth for compliance?
Event window	Critical and error events from chosen logs	Is the window aligned with the incident or review period?
Uptime	Days since last boot	Does this indicate missed maintenance or an intentional workload need?
Listening ports	Bounded list and owning processes	Is this data necessary for the use case and retention policy?

Keep policy honest

Do not tune thresholds only to make the report green. Record why a threshold exists, who owns it, and when it should be reviewed.

08

Configure roles and targets

Use separate target and configuration files when server roles differ materially.

Required services

EXAMPLE IIS ROLE CONFIGURATION

```
"RequiredServices": [  
  "EventLog",  
  "WinRM",  
  "W32Time",  
  "W3SVC"  
]
```

- Use the Windows service name, not only its display name.
- Add a service only when it is expected on every target in that run.
- Keep separate JSON files for domain controllers, IIS, SQL, file servers, or other materially different roles.
- Review the automatic-service exclusion list carefully because trigger-start services can look stopped while behaving normally.

CSV target list

CONFIG\SERVERS.CSV

```
ComputerName,Environment,Owner  
APP-SRV-01,Production,Applications  
SQL-SRV-01,Production,Database Team  
FILE-SRV-01,Test,Infrastructure
```

Required and optional columns

ComputerName is required. Environment and Owner are optional labels that make the HTML and CSV easier to route. Duplicate computer names are removed.

09

Run the first local report

Prove the collector on a non-production Windows host before testing remote access.

Step 1 - Open PowerShell

Use an account permitted to read local system, service, event, registry, and network data.

Step 2 - Run the collector

LOCAL COLLECTION

```
.\Invoke-ServerTriage.ps1 -ComputerName localhost -OpenReport
```

Step 3 - Confirm the result

- A timestamped folder appears beneath Output.
- The HTML report opens in the default browser.
- The console returns report paths and target counts.
- The CSV contains one summary row and the JSON folder contains one host file.
- The log contains a start, collection, completion, and output message.

EXPECTED OUTPUT

```
Output\Run_20260822-143015
|-- Server-Triage-Report.html
|-- Server-Triage-Summary.csv
|-- run-manifest.json
|-- triage-run.log
'-- json\<computer>.json
```

Shortcut

Start-Local-Triage.cmd runs the same local command without bypassing the effective execution policy.

Prepare and test remoting

Remote collection uses the Windows PowerShell remoting path already approved in your environment.

Test the endpoint first

FROM THE COLLECTION COMPUTER

```
Test-WSMan -ComputerName APP-SRV-01
```

If remoting is not configured

Administrative change
On a target approved to receive remote management, Enable-PSRemoting -Force configures WinRM, session endpoints, and firewall rules. Use normal change control.

RUN ON THE APPROVED TARGET AS ADMINISTRATOR

```
Enable-PSRemoting -Force
```

Authentication design

Scenario	Preferred approach
Trusted AD domain	Use the current Windows identity and Kerberos.
Different approved identity	Use Get-Credential and the -Credential parameter.
DMZ, workgroup, or trust boundary	Prefer a correctly named and trusted WinRM HTTPS listener.
Constrained operations	Provide a delegated or Just Enough Administration endpoint where practical.

Avoid
Do not add wildcard TrustedHosts entries or disable certificate checks to make a failed connection disappear.

11

Run against one remote server

The default mode uses the current Windows identity.

Standard domain-connected run

CURRENT IDENTITY

```
.\Invoke-ServerTriage.ps1 -ComputerName APP-SRV-01 -OpenReport
```

What the script does

- Optionally checks ICMP and records the result.
- Attempts Invoke-Command even when ping fails.
- Runs the same read-only collector script block on the target.
- Returns structured objects to the collection computer.
- Applies local threshold logic and writes all output locally.

Skip ICMP when it adds no value

NO PING CHECK

```
.\Invoke-ServerTriage.ps1 -ComputerName APP-SRV-01 -SkipPing
```

An unreachable result is useful evidence

It can point toward name resolution, WinRM, firewall, authentication, endpoint, permission, or host-availability problems. Read triage-run.log for the returned error.

Data location

The collection results are written on the launch computer. The remote script block does not create report files on the target.

12

Run against a server list

CSV mode adds environment and owner context without introducing another dependency.

Create the working list

PREPARE A COPY

```
Copy-Item .\Config\servers.example.csv .\Config\servers.csv
notepad .\Config\servers.csv
```

Run the collection

CSV MODE

```
.\Invoke-ServerTriage.ps1 -ComputerListPath .\Config\servers.csv -OpenReport
```

Use a role-specific configuration

SEPARATE ROLE POLICY

```
.\Invoke-ServerTriage.ps1 -ComputerListPath .\Config\web-servers.csv -ConfigPath .\Config\web-server.config.json
```

Operational notes

- Version 1.0 processes targets sequentially to keep behaviour simple and predictable.
- A failed server does not prevent later targets from being attempted.
- Critical and unreachable results sort above warnings and healthy targets in the HTML.
- Keep each fleet list operationally coherent so required services and thresholds remain meaningful.
- For large estates, start with small batches and measure the effect on WinRM, event logs, and output retention.

Output path

Use -OutputPath C:\TriageEvidence to place run folders in a controlled location with suitable NTFS permissions.

13

Credentials and HTTPS

Keep secrets out of files and use the trust design appropriate to the boundary.

Prompt for an alternate identity

CREDENTIAL HELD IN MEMORY

```
$credential = Get-Credential  
.\Invoke-ServerTriage.ps1 -ComputerName APP-SRV-01 -Credential $credential
```

Use an existing HTTPS listener

HTTPS REMOTING

```
.\Invoke-ServerTriage.ps1 -ComputerName DMZ-SRV-01 -UseSSL -Credential (Get-Credential)
```

Do

Use a valid certificate, matching DNS name, approved listener, least privilege, and controlled output location.

Do not

Embed a password, disable validation, use wildcard trust, email raw credentials, or publish a real report.

Unattended use

- Test interactively with the same identity first.
- Use a managed service account or approved automation identity.
- Grant only required remoting, read, and output permissions.
- Monitor task failure and report freshness.
- Do not use -OpenReport in a scheduled task.

Credential storage is intentionally out of scope

Use the organisation's approved managed identity, credential vault, automation platform, or scheduled-task identity pattern.

What the collector checks

Each area answers a first-pass operational question.

Area	Collected evidence	Question it helps answer
System	OS, build, model, domain, last boot, PowerShell, time zone	Am I looking at the expected host and platform?
Resources	CPU sample, memory use, fixed-disk free space	Is there an obvious point-in-time capacity signal?
Services	Required services and stopped automatic services	Is an expected component missing or not running?
Events	Bounded critical and error events from selected logs	What recent platform or application errors deserve correlation?
Patching	Latest dated hotfixes and common reboot indicators	Is maintenance debt or a pending restart visible?
Network	IPs, gateways, DNS, DHCP, MAC, firewall profiles	Does the host have the expected basic network state?
Listeners	Limited TCP listener list and owning processes	Is the expected process listening locally?
Collection	Section-level exceptions	Was the report complete and what permission or support gap exists?

Performance-conscious event queries
The script uses Get-WinEvent with FilterHashtable, a defined time window, selected levels, and a maximum event count per log.

Known limitation
Get-HotFix is a useful triage signal, not a universal patch-compliance source. Confirm compliance in the organisation's authoritative patch platform.

15

Status logic and interpretation

The highest configured finding becomes the headline status.

Status	Typical trigger	Engineer response
Healthy	No configured threshold crossed	Still validate against the incident and application context.
Warning	Pending reboot, long uptime, low memory snapshot, stopped automatic service, returned error events, or warning-age patch	Review and correlate before deciding on action.
Critical	Critical disk or patch age, missing/stopped required service, or failed core system collection	Prioritise investigation and follow the relevant runbook.
Unreachable	Remote collection failed	Use the error to isolate DNS, WinRM, firewall, authentication, authorisation, endpoint, or availability.

Avoid false certainty

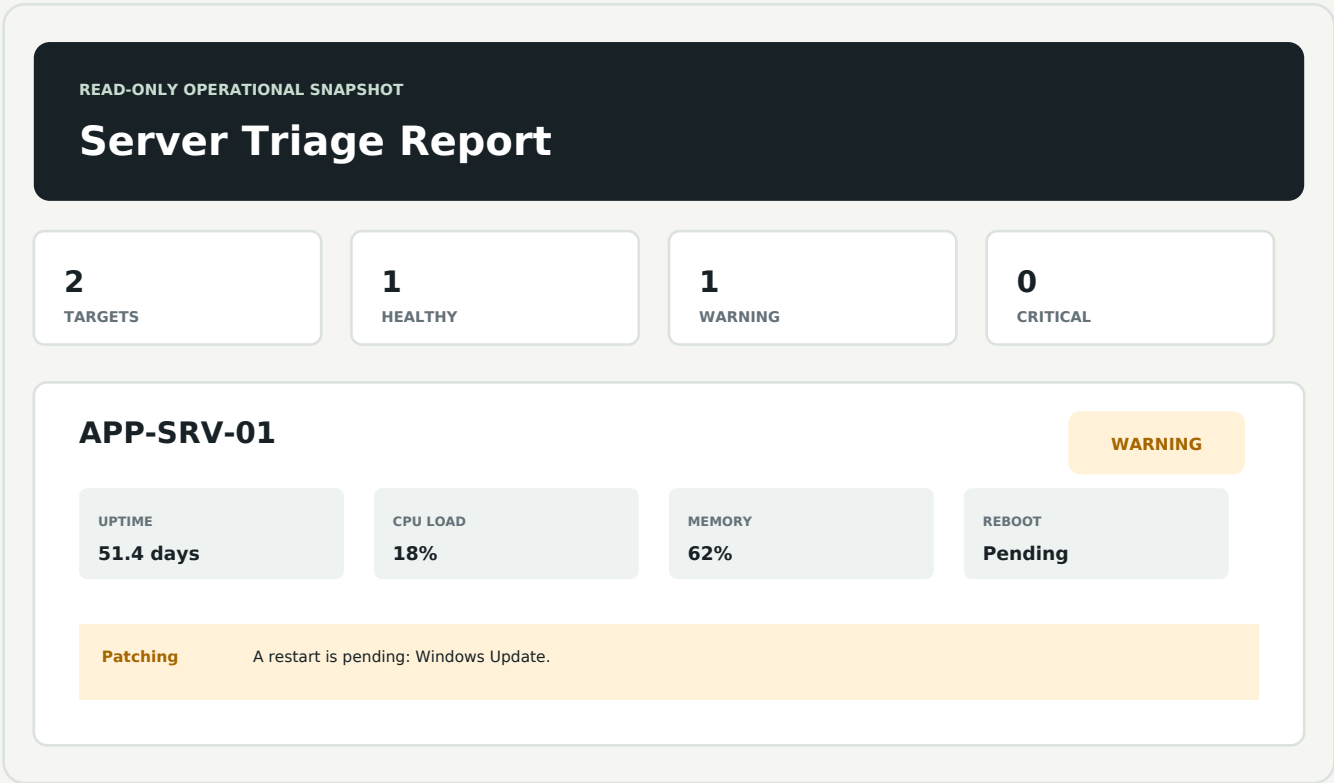
- A CPU value is one sample, not a performance trend.
- Low free memory does not automatically mean harmful memory pressure.
- An event error can be expected noise, a symptom, or unrelated history.
- A stopped automatic service may use trigger start.
- A listening port does not prove end-to-end application health.
- Long uptime can indicate missed maintenance, but it can also be intentional.

Best practice

Correlate the report with monitoring history, application logs, change records, dependency checks, and the reported incident window.

Read the HTML dashboard

The human-facing report gives the summary first and keeps detail expandable.



Section	How to use it
Headline cards	See target counts and overall status distribution.
Server facts	Confirm identity, uptime, resource snapshot, latest patch, and reboot state.
Assessment	Read the threshold findings before opening raw detail.
Disks and services	Prioritise capacity and expected-service issues.
Expandable detail	Inspect events, automatic services, network, listeners, and collection notes.

External sharing
The supplied example report is fictional. Never publish a real output without reviewing and removing sensitive operational data.

17

Use the evidence files

The dashboard is only one view of the result.

CSV summary

Filter by status, owner, environment, patch age, pending reboot, disk free space, event count, or unreachable error.

Host JSON

Compare runs, build drift logic, feed an ITSM workflow, or add a later presentation layer without re-querying the server.

Run manifest

Preserve the run ID, launcher, targets, effective configuration, start and completion times, and expected output.

Execution log

Trace the target sequence, ping context, per-host status, failures, and output locations.

Example follow-on uses

- Attach the HTML and CSV to a controlled incident or problem record.
- Compare the latest JSON with a previous run to detect configuration drift.
- Import the CSV into Power BI or another reporting platform.
- Route only aggregate status into monitoring or workflow automation.
- Retain evidence according to a defined period, then delete it securely.

Preserve context

If evidence enters an incident record, keep the run manifest with it. Thresholds and collection time are necessary to interpret the result later.

18

Troubleshooting

Start with the returned error and test the smallest layer that could explain it.

Symptom	Check	Safe next step
Script blocked	Get-ExecutionPolicy -List	Review the files and use Unblock-File. Follow signing and application-control policy.
WinRM failure	Test-WSMan -ComputerName	Confirm approved remoting configuration, listener, firewall, and name.
Access denied	Identity and endpoint permission	Validate delegated rights, UAC behaviour, CIM, services, event-log, registry, and network reads.
Ping failed	Whether WinRM still connects	Continue or use -SkipPing. ICMP can be blocked.
Partial report	Collection notes and triage-run.log	Identify missing permission, unsupported cmdlet, disabled log, or role-specific condition.
HTTPS trust error	Certificate name, chain, date, and listener	Correct trust. Do not disable validation.

Quick commands

TROUBLESHOOTING SEQUENCE

```
Get-ExecutionPolicy -List
Test-WSMan -ComputerName APP-SRV-01
Resolve-DnsName APP-SRV-01
Test-NetConnection APP-SRV-01 -Port 5985
Get-Content .\Output\Run_<timestamp>\triage-run.log
```

Do not troubleshoot by weakening the control

A bypass, wildcard trust entry, excessive privilege, or disabled validation may hide the symptom while creating a larger security problem.

19

Lab test plan

Use deliberate test cases to prove both the happy path and failure handling.

#	Test case	Expected result
1	Healthy local host	Complete output folder and a readable report
2	Remote host with WinRM	Same data shape as local collection
3	Unresolvable hostname	Unreachable result and useful logged error
4	ICMP blocked, WinRM allowed	Ping warning but successful collection
5	Raised disk warning threshold	Predictable warning without changing disk data
6	Fictional required service	Critical missing-service finding
7	Small event window and limit	Bounded event output
8	Listening-port collection false	No listener rows and no failure
9	Insufficient read permission	Partial or unreachable result with notes
10	Unwritable output path	Clear launch failure and no silent data loss

After each run

- Open the HTML and confirm the displayed state.
- Check the CSV and host JSON agree with the dashboard.
- Review the run log and manifest.
- Restore the standard configuration after forced-threshold tests.
- Record the tested Windows and PowerShell versions.

Validation scope

The included test script checks structure and syntax. It does not replace representative testing on the Windows Server roles you support.

Production hardening

Turn a useful script into a supportable operational tool.

Control area	Production action
Code	Peer review, sign scripts, version control, changelog, controlled release path
Configuration	Role-specific JSON, named owner, approval for threshold or exclusion changes
Identity	Managed or delegated identity, least privilege, constrained endpoint where practical
Transport	Kerberos inside trusted domains; HTTPS where the boundary requires it
Output	Restricted NTFS permissions, classification, retention, secure deletion, backup decision
Operations	Scheduled-task monitoring, report freshness, failure alerts, batch-size testing
Quality	Representative OS and role testing, false-positive review, known-limitations register
Support	Runbook, service owner, escalation path, and planned review date

Recommended release gate

- Validation script passes.
- Lab test cases pass on representative systems.
- No real credentials or operational evidence are inside the project folder.
- Configuration and exclusions are peer reviewed.
- Identity, remoting, output, retention, and ownership are documented.
- Rollback means withdrawing the script or configuration, because collection itself makes no target change.

Separate project
If you later add remediation, build it as an approval-gated companion workflow. Do not quietly convert the evidence collector into a self-healing tool.

21

Extend it safely

The structured output creates a clean route into more advanced infrastructure automation.

Configuration drift

Compare two host JSON files and report only meaningful changes.

ITSM evidence

Attach the HTML or JSON to an incident and preserve the run manifest.

Role adapters

Add separate IIS, SQL, file-server, or domain-controller checks without changing the base contract.

Gated remediation

Route an approved finding into a distinct change-controlled workflow.

Official references

Enable-PSRemoting	https://learn.microsoft.com/powershell/module/microsoft.powershell.core/enable-psremoting
about_Remote_Requirements	https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_remote_requirements
Running Remote Commands	https://learn.microsoft.com/powershell/scripting/security/remoting/running-remote-commands
Get-WinEvent	https://learn.microsoft.com/powershell/module/microsoft.powershell.diagnostics/get-winevent
about_Execution_Policies	https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_execution_policies

Licence and disclaimer

The project is released under the MIT Licence. It is provided as-is. Validate it against your environment, policies, security controls, and change-management requirements before operational use.

Build useful automation. Keep the judgement.